

=== ПРАВИЛА ОБРАЩЕНИЯ С КОДОМ ===

1. правила выдачи кода - если выдача превышает 3500 символов - выдачу необходимо разделять на части.
2. не переименовывай переменные и функции.
3. если нужны другие вызовы функций - используй перегрузку.

=== СТРУКТУРА ВЫБРАННЫХ КАТАЛОГОВ ===

[Файл] Localization\Strings.en.xaml
[Файл] Localization\Strings.ru.xaml
[Файл] Properties\Settings.Designer.cs
[Файл] Properties\Settings.settings
[Файл] Themes\DarkTheme.xaml
[Файл] Themes\LightTheme.xaml
[Файл] App.config
[Файл] App.xaml
[Файл] App.xaml.cs
[Файл] AssemblyInfo.cs
[Файл] ContextBuilderService.cs
[Файл] FileSystemNode.cs
[Файл] FileSystemService.cs
[Файл] GlobalFilterService.cs
[Файл] InverseBooleanConverter.cs
[Файл] MainViewModel.cs
[Файл] MainWindow.xaml
[Файл] MainWindow.xaml.cs
[Файл] models.cs
[Файл] ProjectConfig.cs

=== СОДЕРЖИМОЕ ФАЙЛОВ ===

---Localization\Strings.en.xaml---

```
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:system="clr-namespace:System;assembly=System.Runtime">
<!-- Заголовки блоков -->
<system:String x:Key="Str_ProjManagement">1. Project Management</system:String>
<system:String x:Key="Str_ModuleManagement">2. Slicing Modules / AI Contexts</system:String>
<system:String x:Key="Str_FilesHeader">Current Module Files</system:String>
<system:String x:Key="Str_PromptHeader">General Code Rules (Prompt)</system:String>
```

<!-- Элементы блока проекта -->

```
<system:String x:Key="Str_SelectProj">Select Project:</system:String>
<system:String x:Key="Str_ProjName">Project Name:</system:String>
<system:String x:Key="Str_CreateProj">Create Project</system:String>
<system:String x:Key="Str_Autosave">Autosave on exit</system:String>
<system:String x:Key="Str_DarkTheme">Dark theme</system:String>
<system:String x:Key="Str_RootPath">Root folder (code):</system:String>
<system:String x:Key="Str_OutputPath">Output folder (contexts):</system:String>
<system:String x:Key="Str_Browse">Browse...</system:String>
```

<!-- Элементы блока модулей -->

```
<system:String x:Key="Str_SelectModule">Select Module:</system:String>
<system:String x:Key="Str_ModuleName">Module Name:</system:String>
<system:String x:Key="Str_AddModule">Add Module</system:String>
<system:String x:Key="Str_FileFormat">File Format:</system:String>
```

<!-- Главные кнопки -->

```
<system:String x:Key="Str_SaveProject">SAVE PROJECT CHANGES</system:String>
<system:String x:Key="Str_Generate">GENERATE MODULE CONTEXT FILE</system:String>
```

<!-- Оверлей прогресса -->

```
<system:String x:Key="Str_Cancel">CANCEL OPERATION</system:String>
```

<!-- Фолбэк для заголовка экспандера -->

```
<system:String x:Key="Str_ProjNotSelected">Project not selected</system:String>
<system:String x:Key="Str_AppTitle">Code Context Slicer for AI</system:String>
```

</ResourceDictionary>

---Localization\Strings.ru.xaml---

```
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
```

```

xmlns:system="clr-namespace:System;assembly=System.Runtime">
<!-- Заголовки блоков -->
<system:String x:Key="Str_ProjManagement">1. Управление Проектом</system:String>
<system:String x:Key="Str_ModuleManagement">2. Нарезаемые Модули / Контексты ИИ</system:String>
<system:String x:Key="Str_FilesHeader">Файлы текущего модуля</system:String>
<system:String x:Key="Str_PromptHeader">Общие правила написания кода (Prompt)</system:String>

<!-- Элементы блока проекта -->
<system:String x:Key="Str_SelectProj">Выбор проекта:</system:String>
<system:String x:Key="Str_ProjName">Название проекта:</system:String>
<system:String x:Key="Str_CreateProj">Создать проект</system:String>
<system:String x:Key="Str_Autosave">Автосохранение при выходе</system:String>
<system:String x:Key="Str_DarkTheme">Тёмная тема</system:String>
<system:String x:Key="Str_RootPath">Корневая папка (код):</system:String>
<system:String x:Key="Str_OutputPath">Выходная папка (ТЗ):</system:String>
<system:String x:Key="Str_Browse">Обзор...</system:String>

<!-- Элементы блока модулей -->
<system:String x:Key="Str_SelectModule">Выбор модуля:</system:String>
<system:String x:Key="Str_ModuleName">Название модуля:</system:String>
<system:String x:Key="Str_AddModule">Добавить модуль</system:String>
<system:String x:Key="Str_FileFormat">Формат файла:</system:String>

<!-- Главные кнопки -->
<system:String x:Key="Str_SaveProject">СОХРАНИТЬ ИЗМЕНЕНИЯ ПРОЕКТА</system:String>
<system:String x:Key="Str_Generate">СГЕНЕРИРОВАТЬ ФАЙЛ КОНТЕКСТА МОДУЛЯ</system:String>

<!-- Оверлей прогресса -->
<system:String x:Key="Str_Cancel">ОТМЕНИТЬ ОПЕРАЦИЮ</system:String>

<!-- Фолбэк для заголовка экспандера -->
<system:String x:Key="Str_ProjNotSelected">Проект не выбран</system:String>
<system:String x:Key="Str_AppTitle">Нарезка контекста кода для ИИ</system:String>

</ResourceDictionary>

---Properties\Settings.Designer.cs---
//-----
// <auto-generated>
// Этот код создан программой.
// Исполняемая версия:4.0.30319.42000
//
// Изменения в этом файле могут привести к неправильной работе и будут потеряны в случае
// повторной генерации кода.
// </auto-generated>
//-----

namespace ContextSlicer.Properties {

    [global::System.Runtime.CompilerServices.CompilerGeneratedAttribute()]
    [global::System.CodeDom.Compiler.GeneratedCodeAttribute("Microsoft.VisualStudio.Editors.SettingsDesigner."
    "17.14.0.0")]
    internal sealed partial class Settings : global::System.Configuration.ApplicationSettingsBase {

        private static Settings defaultInstance =
        ((Settings)(global::System.Configuration.ApplicationSettingsBase.Synchronized(new Settings())));

        public static Settings Default {
            get {
                return defaultInstance;
            }
        }

        [global::System.Configuration.UserScopedSettingAttribute()]
        [global::System.Diagnostics.DebuggerNonUserCodeAttribute()]
        [global::System.Configuration.DefaultSettingValueAttribute("True")]
        public bool AutoSaveOnExit {
            get {

```

```

return ((bool)(this["AutoSaveOnExit"]));
}
set {
this["AutoSaveOnExit"] = value;
}
}

```

```

[global::System.Configuration.UserScopedSettingAttribute()]
[global::System.Diagnostics.DebuggerNonUserCodeAttribute()]
[global::System.Configuration.DefaultSettingValueAttribute("True")]
public bool IsDarkTheme {
get {
return ((bool)(this["IsDarkTheme"]));
}
set {
this["IsDarkTheme"] = value;
}
}

```

```

[global::System.Configuration.UserScopedSettingAttribute()]
[global::System.Diagnostics.DebuggerNonUserCodeAttribute()]
[global::System.Configuration.DefaultSettingValueAttribute("True")]
public bool IsProjectBlockExpanded {
get {
return ((bool)(this["IsProjectBlockExpanded"]));
}
set {
this["IsProjectBlockExpanded"] = value;
}
}
}
}

```

```

---Properties\Settings.settings---
<?xml version='1.0' encoding='utf-8'?>
<SettingsFile xmlns="http://schemas.microsoft.com/VisualStudio/2004/01/settings"
CurrentProfile="(Default)" GeneratedClassNamespace="ContextSlicer.Properties"
GeneratedClassName="Settings">
<Profiles />
<Settings>
<Setting Name="AutoSaveOnExit" Type="System.Boolean" Scope="User">
<Value Profile="(Default)">True</Value>
</Setting>
<Setting Name="IsDarkTheme" Type="System.Boolean" Scope="User">
<Value Profile="(Default)">True</Value>
</Setting>
<Setting Name="IsProjectBlockExpanded" Type="System.Boolean" Scope="User">
<Value Profile="(Default)">True</Value>
</Setting>
</Settings>
</SettingsFile>

```

```

---Themes\DarkTheme.xaml---
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
<!-- Цветовая палитра тёмной темы -->
<SolidColorBrush x:Key="WindowBackgroundBrush" Color="#1E1E1E"/>
<SolidColorBrush x:Key="ControlBackgroundBrush" Color="#252526"/>
<SolidColorBrush x:Key="TextBoxBackgroundBrush" Color="#2D2D30"/>
<SolidColorBrush x:Key="TextBrush" Color="#F1F1F1"/>
<SolidColorBrush x:Key="SplitterBrush" Color="#3E3E42"/>
<SolidColorBrush x:Key="BorderBrush" Color="#434346"/>

```

```

<!-- Принудительные стили для BCEX элементов на форме в тёмной теме -->
<Style TargetType="TextBlock">
<Setter Property="Foreground" Value="#DCDCDC"/>
</Style>

```

```

<Style TargetType="GroupBox">
<Setter Property="Foreground" Value="#F1F1F1"/>
<Setter Property="BorderBrush" Value="#3E3E42"/>

```

```
</Style>
```

```
<Style TargetType="CheckBox">  
<Setter Property="Foreground" Value="#DCDCDC"/>  
</Style>
```

```
<Style TargetType="TextBox">  
<Setter Property="Background" Value="{StaticResource TextBoxBackgroundBrush}"/>  
<Setter Property="Foreground" Value="{StaticResource TextBrush}"/>  
<Setter Property="BorderBrush" Value="{StaticResource BorderBrush}"/>  
<Setter Property="BorderThickness" Value="1"/>  
<Setter Property="Padding" Value="3,2"/>  
<Setter Property="CaretBrush" Value="#F1F1F1"/>  
</Style>
```

```
<Style TargetType="TreeView">  
<Setter Property="Background" Value="{StaticResource ControlBackgroundBrush}"/>  
<Setter Property="BorderBrush" Value="{StaticResource BorderBrush}"/>  
</Style>
```

```
<!-- Стилизация ComboBox, чтобы текст внутри него и в выпадающем списке был белым -->  
<Style TargetType="ComboBox">  
<Setter Property="Background" Value="{StaticResource TextBoxBackgroundBrush}"/>  
<Setter Property="Foreground" Value="{StaticResource TextBrush}"/>  
<Setter Property="BorderBrush" Value="{StaticResource BorderBrush}"/>  
</Style>
```

```
<Style TargetType="ComboBoxItem">  
<Setter Property="Background" Value="{StaticResource TextBoxBackgroundBrush}"/>  
<Setter Property="Foreground" Value="{StaticResource TextBrush}"/>  
</Style>
```

```
<!-- Принудительно красим обычные кнопки "Обзор" и "Создать", чтобы они не оставались серыми -->  
<Style TargetType="Button">  
<Setter Property="Background" Value="#2D2D30"/>  
<Setter Property="Foreground" Value="#F1F1F1"/>  
<Setter Property="BorderBrush" Value="#434346"/>  
<Setter Property="Padding" Value="8,2"/>  
</Style>  
</ResourceDictionary>
```

```
---Themes\LightTheme.xaml---  
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">  
<!-- Базовые кисти для светлой темы -->  
<SolidColorBrush x:Key="WindowBackgroundBrush" Color="#F5F5F5"/>  
<SolidColorBrush x:Key="ControlBackgroundBrush" Color="#FFFFFF"/>  
<SolidColorBrush x:Key="TextBoxBackgroundBrush" Color="#FFFFFF"/>  
<SolidColorBrush x:Key="TextBrush" Color="#000000"/>  
<SolidColorBrush x:Key="SplitterBrush" Color="#F0F0F0"/>  
<SolidColorBrush x:Key="BorderBrush" Color="#CCCCCC"/>
```

```
<!-- Неявные стили для сброса на стандартные цвета -->  
<Style TargetType="TextBlock">  
<Setter Property="Foreground" Value="#333333"/>  
</Style>  
<Style TargetType="GroupBox">  
<Setter Property="Foreground" Value="#000000"/>  
<Setter Property="BorderBrush" Value="#CCCCCC"/>  
</Style>  
<Style TargetType="CheckBox">  
<Setter Property="Foreground" Value="#333333"/>  
</Style>  
<Style TargetType="TextBox">  
<Setter Property="Background" Value="{StaticResource TextBoxBackgroundBrush}"/>  
<Setter Property="Foreground" Value="{StaticResource TextBrush}"/>  
</Style>  
<Style TargetType="TreeView">  
<Setter Property="Background" Value="{StaticResource ControlBackgroundBrush}"/>  
</Style>  
</ResourceDictionary>
```

```

---App.config---
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
<configSections>
<sectionGroup name="userSettings" type="System.Configuration.UserSettingsGroup, System,
Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089" >
<section name="ContextSlicer.Properties.Settings" type="System.Configuration.ClientSettingsSection,
System, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089"
allowExeDefinition="MachineToLocalUser" requirePermission="false" />
</sectionGroup>
</configSections>
<userSettings>
<ContextSlicer.Properties.Settings>
<setting name="AutoSaveOnExit" serializeAs="String">
<value>True</value>
</setting>
<setting name="IsDarkTheme" serializeAs="String">
<value>True</value>
</setting>
<setting name="IsProjectBlockExpanded" serializeAs="String">
<value>True</value>
</setting>
</ContextSlicer.Properties.Settings>
</userSettings>
</configuration>

```

```

---App.xaml---
<Application x:Class="ContextSlicer.App"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:local="clr-namespace:ContextSlicer"
StartupUri="MainWindow.xaml">
<Application.Resources>
<ResourceDictionary>
<ResourceDictionary.MergedDictionaries>
<!-- По умолчанию при старте всегда загружаем Светлую тему -->
<ResourceDictionary x:Name="ThemeDictionary">
<SolidColorBrush x:Key="WindowBackgroundBrush" Color="#F5F5F5"/>
<SolidColorBrush x:Key="ControlBackgroundBrush" Color="#FFFFFF"/>
<SolidColorBrush x:Key="TextBoxBackgroundBrush" Color="#FFFFFF"/>
<SolidColorBrush x:Key="TextBrush" Color="#000000"/>
<SolidColorBrush x:Key="SplitterBrush" Color="#E0E0E0"/>
<SolidColorBrush x:Key="BorderBrush" Color="#CCCCCC"/>
</ResourceDictionary>
</ResourceDictionary.MergedDictionaries>
</ResourceDictionary>
</Application.Resources>
</Application>

```

```

---App.xaml.cs---
using System;
using System.Globalization;
using System.Windows;

namespace ContextSlicer
{
    public partial class App : Application
    {
        protected override void OnStartup(StartupEventArgs e)
        {
            base.OnStartup(e);

            // Получаем текущий язык операционной системы Windows
            string currentCulture = CultureInfo.CurrentUICulture.TwoLetterISOLanguageName;

            // Задаем базовое имя файла локализации
            string themeFile = currentCulture.Equals("ru", StringComparison.OrdinalIgnoreCase)
            ? "Strings.ru.xaml"
            : "Strings.en.xaml";

```

```

try
{
var dict = new ResourceDictionary
{
Source = new Uri($"pack://application:,,,/Localization/{themeFile}", UriKind.Absolute)
};

// Внедряем выбранный языковой словарь в глобальные ресурсы приложения
Resources.MergedDictionaries.Add(dict);
}
catch (Exception ex)
{
MessageBox.Show($"Failed to load localization file: {ex.Message}", "Localization Error",
MessageBoxButton.OK, MessageBoxImage.Warning);
}
}
}

---AssemblyInfo.cs---
using System.Windows;

[assembly: ThemeInfo(
ResourceDictionaryLocation.None, //where theme specific resource dictionaries are located
//(used if a resource is not found in the page,
// or application resource dictionaries)
ResourceDictionaryLocation.SourceAssembly //where the generic resource dictionary is located
//(used if a resource is not found in the page,
// app, or any theme specific resource dictionaries)
)]

---ContextBuilderService.cs---
using MigraDoc.Rendering;
using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using System.Threading;
using System.Threading.Tasks;

namespace ContextSlicer;

// Структура для отправки отчетов о прогрессе в UI
public struct ProgressReport
{
public int CurrentIndex { get; set; }
public int TotalCount { get; set; }
public string CurrentFileName { get; set; }
}

public static class ContextBuilderService
{
// Список исключаемых папок (оставляем старый)
private static readonly HashSet<string> ExcludedFolders = new(StringComparer.OrdinalIgnoreCase)
{
"bin", "obj", ".vs", "publish", ".git", ".idea", "node_modules"
};

// Черный список бинарных расширений (чтобы не качать тяжелые файлы по сети)
// Черный список расширений (бинарные данные и мусорные бэкапы/логи)
private static readonly HashSet<string> BinaryExtensions = new(StringComparer.OrdinalIgnoreCase)
{
".png", ".jpg", ".jpeg", ".gif", ".ico", ".bmp", ".webp", // Изображения
".mp3", ".wav", ".ogg", ".flac", ".aac", // Звуки
".mp4", ".avi", ".mkv", ".mov", // Видео
".zip", ".rar", ".7z", ".tar", ".gz", // Архивы
".dll", ".exe", ".pdb", ".suo", ".user", // Бинарники сборки
".fbx", ".obj", ".max", ".blend", ".3ds", // 3D-модели
".bak", ".tmp", ".temp", ".log" // ИСПРАВЛЕНО: Бэкапы, временные файлы и логи
}
}

```

```
};
```

```
// Метод рекурсивного сканирования дерева (оставляем без изменений)
public static FileSystemNode BuildTree(string rootPath, List<string> savedCheckedFiles)
{
    var rootInfo = new DirectoryInfo(rootPath);
    var rootNode = new FileSystemNode { Name = rootInfo.Name, FullPath = rootInfo.FullName,
    RelativePath = "" };
    FillNodesRecursive(rootInfo, rootNode, rootPath, savedCheckedFiles);
    return rootNode;
}
```

```
private static void FillNodesRecursive(DirectoryInfo dir, FileSystemNode parentNode, string
rootPath, List<string> savedCheckedFiles)
{
    try
    {
        foreach (var subDir in dir.GetDirectories())
        {
            if (GlobalFilterService.IsFolderExcluded(subDir.Name)) continue;
            var childNode = new FileSystemNode { Name = subDir.Name, FullPath = subDir.FullName, RelativePath =
            Path.GetRelativePath(rootPath, subDir.FullName), IsFile = false, Parent = parentNode };
            parentNode.Children.Add(childNode);
            FillNodesRecursive(subDir, childNode, rootPath, savedCheckedFiles);
        }
        foreach (var file in dir.GetFiles())
        {
            var relPath = Path.GetRelativePath(rootPath, file.FullName);
            var childNode = new FileSystemNode { Name = file.Name, FullPath = file.FullName, RelativePath =
            relPath, IsFile = true, Parent = parentNode };
            if (savedCheckedFiles.Contains(relPath)) childNode.IsChecked = true;
            parentNode.Children.Add(childNode);
        }
    }
    catch (UnauthorizedAccessException) { }
}
```

```
public static void GetCheckedFiles(FileSystemNode node, List<FileSystemNode> result)
{
    if (node.IsFile && node.IsChecked == true)
    {
        // Проверяем, что файл НЕ является бинарным мусором
        string ext = Path.GetExtension(node.FullPath);
        if (!BinaryExtensions.Contains(ext))
        {
            result.Add(node);
        }
    }
    foreach (var child in node.Children) GetCheckedFiles(child, result);
}
```

```
public static async Task<StringBuilder> BuildTextContentAsync(
    string rules,
    List<FileSystemNode> checkedFiles,
    IProgress<ProgressReport> progress,
    CancellationToken token)
{
    var sb = new StringBuilder();
    sb.AppendLine("=== ПРАВИЛА ОБРАЩЕНИЯ С КОДОМ ===");
    sb.AppendLine(rules ?? ""); // Защита от null
    sb.AppendLine();
```

```
sb.AppendLine("=== СТРУКТУРА ВЫБРАННЫХ КАТАЛОГОВ ===");
    foreach (var file in checkedFiles)
    {
        if (file == null) continue; // Защита от null
        token.ThrowIfCancellationRequested();
        sb.AppendLine($" [Файл] {file.RelativePath ?? ""}");
    }
    sb.AppendLine();
```

```

sb.AppendLine("=== СОДЕРЖИМОЕ ФАЙЛОВ ===");
int total = checkedFiles.Count;

for (int i = 0; i < total; i++)
{
var file = checkedFiles[i];
if (file == null || string.IsNullOrEmpty(file.FullPath)) continue; // Жесткая проверка на null

progress?.Report(new ProgressReport
{
    CurrentIndex = i + 1,
    TotalCount = total,
    CurrentFileName = file.RelativePath ?? ""
});

sb.AppendLine($"---{file.RelativePath ?? ""}---");
try
{
    token.ThrowIfCancellationRequested();
    string content = await File.ReadAllTextAsync(file.FullPath, Encoding.UTF8, token);
    sb.AppendLine(content ?? "");
}
catch (OperationCanceledException) { throw; }
catch (Exception ex)
{
    sb.AppendLine($"[Ошибка чтения файла: {ex.Message}]");
}
sb.AppendLine();
}
return sb;
}

// Асинхронная генерация PDF с поддержкой отмены и прогресса
public static async Task GeneratePdfContextFileAsync(
    string outputPath,
    string fileName,
    string rules,
    FileSystemNode rootNode,
    IProgress<ProgressReport> progress,
    CancellationToken token)
{
    var checkedFiles = new List<FileSystemNode>();
    GetCheckedFiles(rootNode, checkedFiles);

    // Собираем текст асинхронно
    var sb = await BuildTextContentAsync(rules, checkedFiles, progress, token);
    string textData = sb.ToString();

    token.ThrowIfCancellationRequested();

    var document = new MigraDoc.DocumentObjectModel.Document();
    var section = document.AddSection();

    section.PageSetup.TopMargin = MigraDoc.DocumentObjectModel.Unit.FromCentimeter(1);
    section.PageSetup.BottomMargin = MigraDoc.DocumentObjectModel.Unit.FromCentimeter(1);
    section.PageSetup.LeftMargin = MigraDoc.DocumentObjectModel.Unit.FromCentimeter(1);
    section.PageSetup.RightMargin = MigraDoc.DocumentObjectModel.Unit.FromCentimeter(1);

    var style = document.Styles["Normal"];
    if (style != null)
    {
        // Защита от предупреждения CS8602 — свойства изменятся только если стиль успешно найден
        if (style.Font != null)
        {
            style.Font.Name = "Courier New";
            style.Font.Size = 9;
        }
    }
}

```

```

using (var reader = new StringReader(textData))
{
    // Добавили знак вопроса (string?), разрешив переменной принимать null в конце потока
    string? line;
    while ((line = await reader.ReadLineAsync()) != null)
    {
        token.ThrowIfCancellationRequested();
        var paragraph = section.AddParagraph(line);
        if (string.IsNullOrEmpty(line))
        {
            paragraph.Format.SpaceBefore = MigraDoc.DocumentObjectModel.Unit.FromPoint(6);
        }
    }
}

string fullOutputPath = Path.Combine(outputPath, fileName.EndsWith(".pdf") ? fileName : fileName +
    ".pdf");

// Рендеринг PDF запускаем в фоновом потоке, чтобы не вешать UI
await Task.Run(() =>
{
    var renderer = new PdfDocumentRenderer();
    renderer.Document = document;
    renderer.RenderDocument();
    renderer.PdfDocument.Save(fullOutputPath);
}, token);
}

// Асинхронная генерация обычного TXT
public static async Task GenerateContextFileAsync(
    string outputPath,
    string fileName,
    string rules,
    FileSystemNode rootNode,
    IProgress<ProgressReport> progress,
    CancellationToken token)
{
    var checkedFiles = new List<FileSystemNode>();
    GetCheckedFiles(rootNode, checkedFiles);

    var sb = await BuildTextContentAsync(rules, checkedFiles, progress, token);
    string fullOutputPath = Path.Combine(outputPath, fileName.EndsWith(".txt") ? fileName : fileName +
        ".txt");

    await File.WriteAllTextAsync(fullOutputPath, sb.ToString(), Encoding.UTF8, token);
}

}

---FileSystemNode.cs---
using System.Collections.ObjectModel;
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace ContextSlicer;
public class FileSystemNode : INotifyPropertyChanged
{
    private bool? _isChecked = false;
    public string Name { get; set; } = string.Empty;
    public string RelativePath { get; set; } = string.Empty;
    public string FullPath { get; set; } = string.Empty;
    public bool IsFile { get; set; }
    public FileSystemNode? Parent { get; set; }
    public ObservableCollection<FileSystemNode> Children { get; set; } = new();

    public bool? IsChecked
    {

```

```

get => _isChecked;
set => SetChecked(value, true, true);
}

private void SetChecked(bool? value, bool updateChildren, bool updateParent)
{
    if (value == _isChecked) return;
    _isChecked = value;
    OnPropertyChanged(nameof(IsChecked));

    // Если выбрали папку, проставляем статус всем детям внутри
    if (updateChildren && _isChecked.HasValue)
    {
        foreach (var child in Children)
        {
            child.SetChecked(_isChecked, true, false);
        }
    }

    // Обновляем состояние родительской папки (Indeterminate, если выбрано не все)
    if (updateParent && Parent != null)
    {
        Parent.VerifyCheckState();
    }
}

private void VerifyCheckState()
{
    bool? state = null;
    for (int i = 0; i < Children.Count; i++)
    {
        bool? current = Children[i].IsChecked;
        if (i == 0) state = current;
        else if (state != current)
        {
            state = null; // Indeterminate state
            break;
        }
    }
    SetChecked(state, false, true);
}

public event PropertyChangedEventHandler? PropertyChanged;
protected void OnPropertyChanged([CallerMemberName] string? name = null)
=> PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
}

```

---FileSystemService.cs---

```

using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.IO;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ContextSlicer
{
    public class FileSystemService
    {
        private readonly HashSet<string> _excludedDirs = new() { "bin", "obj", ".vs", ".git", ".idea",
            "node_modules", "publish" };

        public async Task<ObservableCollection<FileNode>> ScanDirectoryAsync(string path)
        {
            return await Task.Run(() =>
            {
                var root = new DirectoryInfo(path);
                return new ObservableCollection<FileNode> { ScanRecursive(root) };
            });
        }
    }
}

```

```

}

private FileNode ScanRecursive(DirectoryInfo dir)
{
    var node = new FileNode
    {
        Name = dir.Name,
        FullPath = dir.FullName,
        IsDirectory = true
    };

    try
    {
        foreach (var subDir in dir.GetDirectories().Where(d => !_excludedDirs.Contains(d.Name.ToLower())))
        {
            node.Children.Add(ScanRecursive(subDir));
        }

        foreach (var file in dir.GetFiles())
        {
            node.Children.Add(new FileNode
            {
                Name = file.Name,
                FullPath = file.FullName,
                IsDirectory = false
            });
        }
    }
    catch (UnauthorizedAccessException) { /* Логгирование или пропуск */ }

    return node;
}
}
}
}

```

```

---GlobalFilterService.cs---
using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.IO;
using System.Text.Json;

namespace ContextSlicer;

public class GlobalFilters
{
    public ObservableCollection<string> ExcludedFolders { get; set; } = new();
    public ObservableCollection<string> ExcludedExtensions { get; set; } = new();
}

public static class GlobalFilterService
{
    {
        private static readonly string FilePath;
        public static GlobalFilters Current { get; private set; } = new();

        static GlobalFilterService()
        {
            string appData = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData);
            string appDir = Path.Combine(appData, "ContextSlicer");
            Directory.CreateDirectory(appDir);
            FilePath = Path.Combine(appDir, "global_filters.json");

            LoadFilters();
        }

        public static void LoadFilters()
        {
            try

```

```

{
if (File.Exists(FilePath))
{
string json = File.ReadAllText(FilePath);
var loaded = JsonSerializer.Deserialize<GlobalFilters>(json);
if (loaded != null)
{
Current = loaded;
return;
}
}
}
catch { }

// Если файла нет или он поврежден — накатываем базовый дефолтный список
Current = new GlobalFilters
{
ExcludedFolders = new ObservableCollection<string> { "bin", "obj", ".vs", "publish", ".git",
".idea", "node_modules" },
ExcludedExtensions = new ObservableCollection<string> { ".png", ".jpg", ".jpeg", ".gif", ".ico",
".bmp", ".webp", ".mp3", ".wav", ".zip", ".rar", ".dll", ".exe", ".pdb", ".suo", ".user", ".bak",
".tmp", ".log" }
};
SaveFilters();
}

public static void SaveFilters()
{
try
{
string json = JsonSerializer.Serialize(Current, new JsonSerializerOptions { WriteIndented = true
});
File.WriteAllText(FilePath, json);
}
catch { }
}

// Теперь проверки папок и расширений работают на основе глобальных ObservableCollection
динамически
public static bool IsFolderExcluded(string folderName)
{
foreach (var f in GlobalFilterService.Current.ExcludedFolders)
{
if (folderName.Equals(f, StringComparison.OrdinalIgnoreCase)) return true;
}
return false;
}

public static bool IsExtensionExcluded(string filePath)
{
string ext = Path.GetExtension(filePath);
foreach (var e in GlobalFilterService.Current.ExcludedExtensions)
{
if (ext.Equals(e, StringComparison.OrdinalIgnoreCase)) return true;
}
return false;
}

}

```

---InverseBooleanConverter.cs---

```

using System;
using System.Globalization;
using System.Windows.Data;

namespace ContextSlicer;

[ValueConversion(typeof(bool), typeof(bool))]
public class InverseBooleanConverter : IValueConverter
{

```

```

public static InverseBooleanConverter Instance { get; } = new InverseBooleanConverter();

public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
{
    if (value is bool b) return !b;
    return false;
}

public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
{
    if (value is bool b) return !b;
    return false;
}
}

---MainViewModel.cs---
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using Microsoft.WindowsAPICodePack.Dialogs;
using Newtonsoft.Json;
using PdfSharp.Fonts;
using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.IO;
using System.Linq;
using System.Reflection;
using System.Windows;
using System.Windows.Media;
using MigraDoc;

namespace ContextSlicer;

public partial class MainViewModel : ObservableObject
{
    private readonly string _configFilePath;
    [ObservableProperty] private bool _isDarkTheme;
    [ObservableProperty] private ObservableCollection<ProjectConfig> _projects = new();
    [ObservableProperty] private ProjectConfig? _selectedProject;
    [ObservableProperty] private bool _isAutoSaveEnabled;
    // Список модулей выбранного проекта
    [ObservableProperty] private bool _isPdfFormat; // Если true – рендерим в PDF, если false – в TXT
    [ObservableProperty] private bool _isProjectBlockExpanded;

    [ObservableProperty] private FileSystemNode? _rootNode;

    // Поля ввода UI
    [ObservableProperty] private string _projectNameInput = string.Empty;
    [ObservableProperty] private string _rootPath = string.Empty;
    [ObservableProperty] private string _outputPath = string.Empty;
    [ObservableProperty] private string _promptRules = string.Empty;

    [ObservableProperty] private string _moduleNameInput = string.Empty;
    [ObservableProperty] private string _contextFileName = string.Empty;
    [ObservableProperty]
    private ObservableCollection<ContextModule> _modules = new();

    // Внимательно проверьте написание этой переменной:
    [ObservableProperty]
    private ContextModule? _selectedModule;
    // Свойство доступности блока модулей (теперь со строгим уведомлением для интерфейса)
    // Настраиваемый черный список папок (дефолтные значения)
    [ObservableProperty]
    private string _excludedFoldersInput = "bin, obj, .vs, publish, .git, .idea, node_modules";

    // Настраиваемый черный список мусорных и бинарных расширений (дефолтные значения)
    [ObservableProperty]

```

```

private string _excludedExtensionsInput = ".png, .jpg, .jpeg, .gif, .ico, .bmp, .webp, .mp3, .wav,
.ogg, .flac, .aac, .mp4, .avi, .mkv, .mov, .zip, .rar, .7z, .tar, .gz, .dll, .exe, .pdb, .suo,
.user, .fbx, .obj, .max, .blend, .3ds, .bak, .tmp, .temp, .log";

[RelayCommand]
private void DeleteCurrentModule()
{
    if (SelectedModule == null) return;

    // Спрашиваем подтверждение удаления на языке системы
    var result = System.Windows.MessageBox.Show(
        "Вы уверены, что хотите полностью удалить этот модуль и все его сохраненные настройки файлов?",
        "Удаление модуля",
        MessageBoxButton.YesNo,
        MessageBoxImage.Question);

    if (result == MessageBoxResult.Yes)
    {
        var moduleToRemove = SelectedModule;
        SelectedModule = null; // Сбрасываем выбор

        Modules.Remove(moduleToRemove);
        OnPropertyChanged(nameof(IsModuleSelectorEnabled));

        // Автоматически сохраняем изменения на диск
        SaveProject();
    }

    public bool IsModuleSelectorEnabled
    {
        get => SelectedProject != null;
    }

    // Свойство для динамического вывода имени проекта в заголовок Expander
    // Динамическое свойство для заголовка экспандера (Исключает наложение текста)
    public string DisplayProjectName
    {
        get
        {
            // Если проект выбран и у него есть имя — выводим его
            if (SelectedProject != null && !string.IsNullOrEmpty(SelectedProject.ProjectName))
            {
                return SelectedProject.ProjectName;
            }

            // Если проект не выбран — безопасно вытаскиваем локализованную строку из ресурсов окна
            if (Application.Current?.MainWindow?.Resources != null &&
                Application.Current.MainWindow.Resources.Contains("Str_ProjNotSelected"))
            {
                return Application.Current.MainWindow.Resources["Str_ProjNotSelected"] as string ?? "---";
            }

            // Фолбэк на случай, если ресурсы еще не успели загрузиться при самом первом старте
            return "---";
        }
    }

    public MainViewModel()
    {
        // Точный и правильный путь к настройке шрифтов Windows в PDFsharp/MigraDoc
        PdfSharp.Fonts.GlobalFontSettings.UseWindowsFontsUnderWindows = true;

        // Оставляем провайдер кодировок для кириллицы
        System.Text.Encoding.RegisterProvider(System.Text.CodePagesEncodingProvider.Instance);

        // ... остальной код конструктора без изменений ...
        string appData = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData);

```

```

string appDir = Path.Combine(appData, "ContextSlicer");
Directory.CreateDirectory(appDir);
_configFilePath = Path.Combine(appDir, "projects.json");

IsAutoSaveEnabled = Properties.Settings.Default.AutoSaveOnExit;
IsDarkTheme = Properties.Settings.Default.IsDarkTheme;
IsProjectBlockExpanded = Properties.Settings.Default.IsProjectBlockExpanded;
OnIsDarkThemeChanged(IsDarkTheme);

LoadProjects();
}

// Логика при выборе ПРОЕКТА
partial void OnSelectedProjectChanged(ProjectConfig? value)
{
    if (value != null)
    {
        ProjectNameInput = value.ProjectName;
        RootPath = value.RootPath;
        OutputPath = value.OutputPath;
        PromptRules = value.PromptRules;

        // Загружаем список модулей этого проекта
        Modules = new ObservableCollection<ContextModule>(value.Modules);

        // Перестраиваем дерево папок базово (без выбранных файлов)
        RefreshTreeView(new List<string>());

        // Выбираем первый модуль, если он есть
        SelectedModule = Modules.FirstOrDefault();
    }
    else
    {
        Modules.Clear();
        RootNode = null;
    }
    OnPropertyChanged(nameof(IsModuleSelectorEnabled));
    OnPropertyChanged(nameof(DisplayProjectName));
}

// Логика при выборе конкретного МОДУЛЯ (контекста) внутри проекта
// Имя должно быть ОДИН В ОДИН как имя свойства после On...
partial void OnSelectedModuleChanged(ContextModule? value)
{
    if (value != null)
    {
        ModuleNameInput = value.ModuleName;
        ContextFileName = value.ContextFileName;

        // Перерисовываем дерево и проставляем галочки, сохраненные для ЭТОГО модуля
        RefreshTreeView(value.CheckedFiles);
    }
    else
    {
        ModuleNameInput = string.Empty;
        ContextFileName = string.Empty;
        RefreshTreeView(new List<string>());
    }
}

private void RefreshTreeView(List<string> checkedFiles)
{
    if (Directory.Exists(RootPath))
    {
        RootNode = ContextBuilderService.BuildTree(RootPath, checkedFiles);
    }
}

```

```

else
{
RootNode = null;
}
}

[RelayCommand]
private void CreateNewProject()
{
// Исправлено: если имя введено в поле — берем его, иначе даем дефолтное
string name = string.IsNullOrEmpty(ProjectNameInput)
? $"Проект {Projects.Count + 1}"
: ProjectNameInput;

var newProject = new ProjectConfig { ProjectName = name };
Projects.Add(newProject);
SelectedProject = newProject;
}

[RelayCommand]

private void CreateNewModule()
{
if (SelectedProject == null)
{
System.Windows.MessageBox.Show("Сначала выберите или создайте проект!", "Внимание",
MessageBoxButton.OK, MessageBoxImage.Warning);
return;
}

string name = string.IsNullOrEmpty(ModuleNameInput)
? $"Модуль {Modules.Count + 1}"
: ModuleNameInput;

var newModule = new ContextModule
{
ModuleName = name,
ContextFileName = $"{GetSafeFileName(name)}.txt"
};

Modules.Add(newModule);
SelectedModule = newModule;

// СВРОС ЧЕКБОКСОВ: Принудительно очищаем дерево папок для нового модуля
RefreshTreeView(new List<string>());
}

// Изменим сигнатуру метода, добавив необязательный параметр showMessage

[RelayCommand]
public void SaveProject()
{
ExecuteSave(showMessage: true);
}

// Этот метод мы вызываем при закрытии, он работает без всплывающих окон
public void SilentSave()
{
ExecuteSave(showMessage: false);
}

// Основная логика сохранения
private void ExecuteSave(bool showMessage)
{
if (SelectedProject == null)
{
if (!string.IsNullOrEmpty(ProjectNameInput))
{

```

```

var autoProject = new ProjectConfig { ProjectName = ProjectNameInput };
Projects.Add(autoProject);
SelectedProject = autoProject;
}
else
{
    if (showMessage) System.Windows.MessageBox.Show("Введите имя проекта перед сохранением.",
        "Внимание", MessageBoxButton.OK, MessageBoxImage.Warning);
    return;
}
}

if (SelectedModule != null && RootNode != null)
{
    var checkedFilesList = new List<string>();
    var flatList = new List<FileSystemNode>();
    ContextBuilderService.GetCheckedFiles(RootNode, flatList);
    foreach (var node in flatList)
    {
        checkedFilesList.Add(node.RelativePath);
    }

    SelectedModule.ModuleName = ModuleNameInput;
    SelectedModule.ContextFileName = GetSafeFileName(ModuleNameInput) + ".txt";
    SelectedModule.CheckedFiles = checkedFilesList;

    var mIdx = Modules.IndexOf(SelectedModule);
    if (mIdx >= 0) Modules[mIdx] = SelectedModule;
}

SelectedProject.ProjectName = ProjectNameInput;
SelectedProject.RootPath = RootPath;
SelectedProject.OutputPath = OutputPath;
SelectedProject.PromptRules = PromptRules;
SelectedProject.Modules = Modules.ToList();

var pIdx = Projects.IndexOf(SelectedProject);
if (pIdx >= 0)
{
    Projects[pIdx] = SelectedProject;
    SelectedProject = Projects[pIdx];
}

try
{
    string json = JsonConvert.SerializeObject(Projects, Formatting.Indented);
    File.WriteAllText(_configFilePath, json);
    if (showMessage) System.Windows.MessageBox.Show("Всё успешно сохранено!", "Успех",
        MessageBoxButton.OK, MessageBoxImage.Information);
}
catch (Exception ex)
{
    if (showMessage) System.Windows.MessageBox.Show($"Ошибка сохранения JSON: {ex.Message}", "Ошибка",
        MessageBoxButton.OK, MessageBoxImage.Error);
}
}

[RelayCommand]
private void BrowseRootPath()
{
    var dialog = new CommonOpenFileDialog { IsFolderPicker = true };
    if (dialog.ShowDialog() == CommonFileDialogResult.Ok)
    {
        RootPath = dialog.FileName;
        RefreshTreeView(SelectedModule?.CheckedFiles ?? new List<string>());
    }
}

[RelayCommand]
private void BrowseOutputPath()

```

```

{
var dialog = new CommonOpenFileDialog { IsFolderPicker = true };
if (dialog.ShowDialog() == CommonFileDialogResult.Ok)
{
OutputPath = dialog.FileName;
}
}

private void LoadProjects()
{
if (File.Exists(_configFilePath))
{
try
{
string json = File.ReadAllText(_configFilePath);
var list = JsonConvert.DeserializeObject<List<ProjectConfig>>(json);
if (list != null)
{
Projects = new ObservableCollection<ProjectConfig>(list);
if (Projects.Count > 0) SelectedProject = Projects[0];
}
}
catch { }
}
}

// Метод автоматической генерации безопасного имени файла
private string GetSafeFileName(string name)
{
if (string.IsNullOrEmpty(name)) return "context";

string safeName = name.Replace(" ", "-");
char[] invalidChars = Path.GetInvalidFileNameChars();

foreach (char c in invalidChars)
{
safeName = safeName.Replace(c.ToString(), "");
}
return string.IsNullOrEmpty(safeName) ? "context" : safeName.ToLower();
}

// Метод, вызываемый при закрытии приложения
public void OnWindowClosing()
{
// Сохраняем состояние темы в настройки по умолчанию
Properties.Settings.Default.IsDarkTheme = IsDarkTheme;
// Сохраняем состояние чекбокса в Settings приложения
Properties.Settings.Default.AutoSaveOnExit = IsAutoSaveEnabled;
Properties.Settings.Default.IsProjectBlockExpanded = IsProjectBlockExpanded;
Properties.Settings.Default.Save();

// Если чекбокс активен – сохраняем проект без вывода MessageBox
if (IsAutoSaveEnabled)
{
SilentSave();
}
}

// Этот метод срабатывает автоматически каждый раз, когда меняется галочка Тёмной темы
partial void OnIsDarkThemeChanged(bool value)
{
// Очищаем старую палитру приложения
Application.Current.Resources.MergedDictionaries.Clear();

var themeDict = new ResourceDictionary();

if (value)
{
// Накатываем каноничные цвета глубокой тёмной темы VS Code
themeDict.Add("WindowBackgroundBrush", new SolidColorBrush(Color.FromRgb(0x1E, 0x1E, 0x1E)));
}
}

```

```

themeDict.Add("ControlBackgroundBrush", new SolidColorBrush(Color.FromRgb(0x25, 0x25, 0x26)));
themeDict.Add("TextBoxBackgroundBrush", new SolidColorBrush(Color.FromRgb(0x2D, 0x2D, 0x30)));
themeDict.Add("TextBrush", new SolidColorBrush(Color.FromRgb(0xF1, 0xF1, 0xF1)));
themeDict.Add("SplitterBrush", new SolidColorBrush(Color.FromRgb(0x3E, 0x3E, 0x42)));
themeDict.Add("BorderBrush", new SolidColorBrush(Color.FromRgb(0x43, 0x43, 0x46)));
}
else
{
    // Возвращаем чистые светлые цвета Windows
    themeDict.Add("WindowBackgroundBrush", new SolidColorBrush(Color.FromRgb(0xF5, 0xF5, 0xF5)));
    themeDict.Add("ControlBackgroundBrush", new SolidColorBrush(Color.FromRgb(0xFF, 0xFF, 0xFF)));
    themeDict.Add("TextBoxBackgroundBrush", new SolidColorBrush(Color.FromRgb(0xFF, 0xFF, 0xFF)));
    themeDict.Add("TextBrush", new SolidColorBrush(Color.FromRgb(0x00, 0x00, 0x00)));
    themeDict.Add("SplitterBrush", new SolidColorBrush(Color.FromRgb(0xE0, 0xE0, 0xE0)));
    themeDict.Add("BorderBrush", new SolidColorBrush(Color.FromRgb(0xCC, 0x2C, 0xCC)));
}

// Внедряем новую палитру в глобальный контекст WPF
Application.Current.Resources.MergedDictionaries.Add(themeDict);
}

// Маленький служебный класс-заглушка для связки MigraDoc с системными шрифтами
private CancellationTokenSource? _cts;

// Свойства для отображения оверлея прогресса
[ObservableProperty] private bool _isProcessing;
[ObservableProperty] private int _progressValue;
[ObservableProperty] private int _progressMax;
[ObservableProperty] private string _progressText = string.Empty;
[ObservableProperty] private string _currentFileText = string.Empty;

// Команда "Отмена"
[RelayCommand]
private void CancelGeneration()
{
    _cts?.Cancel();
    ProgressText = "Отмена операции...";
}

// Полностью заменяем старую команду GenerateContext на асинхронную
[RelayCommand]
private async Task GenerateContext()
{
    string extension = IsPdfFormat ? ".pdf" : ".txt";
    string safeFileName = GetSafeFileName(ModuleNameInput) + extension;

    if (string.IsNullOrEmpty(OutputPath) || string.IsNullOrEmpty(ModuleNameInput) || RootNode
        == null)
    {
        System.Windows.MessageBox.Show("Не заполнены критические данные: выходная папка, имя модуля или структура проекта.", "Ошибка", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    var checkedFiles = new List<FileSystemNode>();
    ContextBuilderService.GetCheckedFiles(RootNode, checkedFiles);
    if (checkedFiles.Count == 0)
    {
        System.Windows.MessageBox.Show("Не выбрано ни одного файла для нарезки контекста.", "Внимание", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    // Включаем оверлей прогресса в UI
    IsProcessing = true;
    ProgressValue = 0;
    ProgressMax = checkedFiles.Count;
    ProgressText = $"Подготовка к обработке {checkedFiles.Count} файлов...";
    CurrentFileText = "";

```

```

_cts = new CancellationTokenSource();

// Настраиваем отправку прогресса в поток UI
var progressHandler = new Progress<ProgressReport>(report =>
{
    ProgressValue = report.CurrentIndex;
    ProgressText = $"Обработано файлов: {report.CurrentIndex} из {report.TotalCount}";
    CurrentFileText = $"Текущий файл: {report.CurrentFileName}";
});

try
{
    string fullPath = Path.Combine(OutputPath, safeFileName);

    if (IsPdfFormat)
    {
        await ContextBuilderService.GeneratePdfContextFileAsync(OutputPath, safeFileName, PromptRules,
            RootNode, progressHandler, _cts.Token);
    }
    else
    {
        await ContextBuilderService.GenerateContextFileAsync(OutputPath, safeFileName, PromptRules,
            RootNode, progressHandler, _cts.Token);
    }

    // Автооткрытие Проводника Windows (оставляем без изменений)
    if (File.Exists(fullPath))
    {
        System.Diagnostics.Process.Start("explorer.exe", $"/select,\"{fullPath}\"");
    }
    catch (OperationCanceledException)
    {
        System.Windows.MessageBox.Show("Операция сборки контекста была отменена пользователем.", "Отмена",
            MessageBoxButton.OK, MessageBoxImage.Information);
    }
    catch (Exception ex)
    {
        System.Windows.MessageBox.Show($"Ошибка сборки файла: {ex.Message}", "Ошибка", MessageBoxButton.OK,
            MessageBoxImage.Error);
    }
    finally
    {
        // Выключаем оверлей в любом случае
        IsProcessing = false;
        _cts?.Dispose();
        _cts = null;
    }
}

```

```

---MainWindow.xaml---
<Window x:Class="ContextSlicer.MainWindow"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
xmlns:local="clr-namespace:ContextSlicer"
mc:Ignorable="d"
Title="{DynamicResource Str_AppTitle}" Height="450" Width="780"
MinWidth="780" MinHeight="450"
WindowStartupLocation="CenterScreen"
Closing="Window_Closing"
Background="{DynamicResource WindowBackgroundBrush}"
Foreground="{DynamicResource TextBrush}">

```

```

<Window.Resources>
<ResourceDictionary>
<ResourceDictionary.MergedDictionaries>

```

```

<!-- Подгружаем дефолтный словарь, чтобы дизайнер Студии сразу видел все надписи -->
<ResourceDictionary Source="pack://application:,,,/Localization/Strings.ru.xaml"/>
</ResourceDictionary.MergedDictionaries>
<!-- 1. Текстовые блоки (метки) -->
<Style TargetType="TextBlock">
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
</Style>

<!-- 2. Стилизация рамок GroupBox -->
<Style TargetType="GroupBox">
<Setter Property="OverridesDefaultStyle" Value="True"/>
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
<Setter Property="Background" Value="{DynamicResource ControlBackgroundBrush}"/>
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="GroupBox">
<Border Background="{DynamicResource ControlBackgroundBrush}"
BorderBrush="{DynamicResource BorderBrush}" BorderThickness="1" CornerRadius="3" Padding="5">
<Grid Background="{DynamicResource ControlBackgroundBrush}">
<Grid.RowDefinitions>
<RowDefinition Height="Auto"/>
<RowDefinition Height="*/"/>
</Grid.RowDefinitions>
<Border BorderThickness="0,0,0,1" BorderBrush="{DynamicResource BorderBrush}" Padding="0,0,0,4"
Margin="0,0,0,5">
<ContentPresenter ContentSource="Header" TextBlock.FontWeight="Bold"/>
</Border>
<ContentPresenter Grid.Row="1"/>
</Grid>
</Border>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>

<!-- 3. Чекбоксы и Радиокнопки -->
<Style TargetType="CheckBox">
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Setter Property="VerticalAlignment" Value="Center"/>
</Style>
<Style TargetType="RadioButton">
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Setter Property="VerticalAlignment" Value="Center"/>
<Setter Property="Margin" Value="0,0,15,0"/>
</Style>

<!-- 4. Текстовые поля ввода -->
<Style TargetType="TextBox">
<Setter Property="Background" Value="{DynamicResource TextBoxBackgroundBrush}"/>
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
<Setter Property="BorderThickness" Value="1"/>
<Setter Property="Padding" Value="4,2"/>
<Setter Property="CaretBrush" Value="{DynamicResource TextBrush}"/>
</Style>

<!-- 5. Дерево файлов (TreeView) -->
<Style TargetType="TreeView">
<Setter Property="Background" Value="{DynamicResource ControlBackgroundBrush}"/>
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
<Setter Property="BorderThickness" Value="1"/>
</Style>
<Style TargetType="TreeViewItem">
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
</Style>

<!-- 6. Мелкие серые кнопки управления -->
<Style TargetType="Button">
<Setter Property="Background" Value="{DynamicResource TextBoxBackgroundBrush}"/>
<Setter Property="Foreground" Value="{DynamicResource TextBrush}"/>
<Setter Property="BorderBrush" Value="{DynamicResource BorderBrush}"/>
<Setter Property="Padding" Value="10,2"/>
<Setter Property="Height" Value="24"/>

```

```

</Style>
<BooleanToVisibilityConverter x:Key="BooleanToVisibilityConverter"/>
</ResourceDictionary>
</Window.Resources>
<!-- Главный контейнер формы с принудительной динамической кистью фона -->
<!-- Главный контейнер формы -->
<Grid Margin="10" Background="{DynamicResource WindowBackgroundBrush}">
<Grid.RowDefinitions>
<RowDefinition Height="Auto"/>
<RowDefinition Height="Auto"/>
<RowDefinition Height="*/>
<RowDefinition Height="Auto"/>
</Grid.RowDefinitions>

<!-- 1. УПРАВЛЕНИЕ ПРОЕКТОМ (Expander с мультиязычным заголовком) -->
<Expander Grid.Row="0" ExpandDirection="Down" Margin="0,0,0,10"
IsExpanded="{Binding IsProjectBlockExpanded}">
<Expander.Header>
<StackPanel Orientation="Horizontal">
<!-- Основное название блока -->
<TextBlock Text="{DynamicResource Str_ProjManagement}" FontWeight="Bold" FontSize="12"/>
<TextBlock Text=": " FontWeight="Bold" FontSize="12"/>

<!-- Чистая и безопасная строка статуса проекта без наложений -->
<TextBlock Text="{Binding DisplayProjectName}" FontWeight="Bold" FontSize="12"
Foreground="#007ACC"/>
</StackPanel>
</Expander.Header>

<Grid Margin="0,5,0,0" Background="{DynamicResource ControlBackgroundBrush}">
<Grid.RowDefinitions>
<RowDefinition Height="Auto"/>
<RowDefinition Height="Auto"/>
<RowDefinition Height="Auto"/>
</Grid.RowDefinitions>

<WrapPanel Grid.Row="0" Orientation="Horizontal" VerticalAlignment="Center" Margin="0,0,0,8">
<StackPanel Orientation="Horizontal" Margin="0,4,15,4">
<TextBlock Text="{DynamicResource Str_SelectProj}" VerticalAlignment="Center" Margin="5,0,5,0"/>
<ComboBox ItemsSource="{Binding Projects}"
SelectedItem="{Binding SelectedProject}"
DisplayMemberPath="ProjectName"
Width="200" Height="26" VerticalContentAlignment="Center"/>
</StackPanel>

<StackPanel Orientation="Horizontal" Margin="0,4,15,4">
<TextBlock Text="{DynamicResource Str_ProjName}" VerticalAlignment="Center" Margin="0,0,5,0"/>
<TextBox Text="{Binding ProjectNameInput, UpdateSourceTrigger=PropertyChanged}" Width="180"
Height="24" VerticalContentAlignment="Center"/>
</StackPanel>

<Button Content="{DynamicResource Str_CreateProj}" Command="{Binding CreateNewProjectCommand}"
Margin="0,4,15,4" Height="26"/>
<CheckBox IsChecked="{Binding IsAutoSaveEnabled}" Content="{DynamicResource Str_Autosave}"
Margin="0,4,15,4"/>
<CheckBox IsChecked="{Binding IsDarkTheme}" Content="{DynamicResource Str_DarkTheme}"
Margin="10,4,15,4"/>
</WrapPanel>

<!-- Пути к папкам проекта -->
<Grid Grid.Row="1" Margin="5,0,5,6">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="150"/>
<ColumnDefinition Width="*/>
<ColumnDefinition Width="Auto"/>
</Grid.ColumnDefinitions>
<TextBlock Text="{DynamicResource Str_RootPath}" VerticalAlignment="Center" FontWeight="SemiBold"/>
<TextBox Grid.Column="1" Text="{Binding RootPath, UpdateSourceTrigger=PropertyChanged}" Height="24"
VerticalContentAlignment="Center" IsReadOnly="True"/>
<Button Grid.Column="2" Content="{DynamicResource Str_Browse}" Command="{Binding
BrowseRootPathCommand}" Width="75" Margin="5,0,0,0"/>

```

```
</Grid>
```

```
<Grid Grid.Row="2" Margin="5,0,5,4">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="150"/>
<ColumnDefinition Width="*/>
<ColumnDefinition Width="Auto"/>
</Grid.ColumnDefinitions>
<TextBlock Text="{DynamicResource Str_OutputPath}" VerticalAlignment="Center"
FontWeight="SemiBold"/>
<TextBox Grid.Column="1" Text="{Binding OutputPath, UpdateSourceTrigger=PropertyChanged}"
Height="24" VerticalContentAlignment="Center" IsReadOnly="True"/>
<Button Grid.Column="2" Content="{DynamicResource Str_Browse}" Command="{Binding
BrowseOutputPathCommand}" Width="75" Margin="5,0,0,0"/>
</Grid>
</Grid>
</Expander>
<!-- 2. БЛОК УПРАВЛЕНИЯ МОДУЛЯМИ -->
<GroupBox Header="{DynamicResource Str_ModuleManagement}" Grid.Row="1" Margin="0,0,0,15"
Padding="5">
<Grid>
<Grid.ColumnDefinitions>
<ColumnDefinition Width="Auto"/>
<ColumnDefinition Width="*/>
<ColumnDefinition Width="Auto"/>
</Grid.ColumnDefinitions>

<StackPanel Orientation="Horizontal" Grid.Column="0" VerticalAlignment="Top" Margin="0,2,0,0">
<TextBlock Text="{DynamicResource Str_SelectModule}" VerticalAlignment="Center" Margin="5,0,5,0"/>
<ComboBox ItemsSource="{Binding Modules}"
SelectedItem="{Binding SelectedModule}"
DisplayMemberPath="ModuleName"
Width="200" Height="26" VerticalContentAlignment="Center"
IsEnabled="{Binding IsModuleSelectorEnabled}"/>
</StackPanel>
```

```
<Grid Grid.Column="1" Margin="20,0,20,0">
<Grid.RowDefinitions>
<RowDefinition Height="Auto"/>
<RowDefinition Height="Auto"/>
</Grid.RowDefinitions>
```

```
<StackPanel Orientation="Horizontal" Grid.Row="0" Margin="0,0,0,6">
<TextBlock Text="{DynamicResource Str_ModuleName}" VerticalAlignment="Center" Width="110"/>
<TextBox Text="{Binding ModuleNameInput, UpdateSourceTrigger=PropertyChanged}" Width="220"
Height="24" VerticalContentAlignment="Center"/>
</StackPanel>
```

```
<StackPanel Orientation="Horizontal" Grid.Row="1" Margin="0,2,0,0">
<TextBlock Text="{DynamicResource Str_FileFormat}" VerticalAlignment="Center" Width="110"
FontWeight="SemiBold"/>
<RadioButton Content="txt" IsChecked="{Binding IsPdfFormat, Converter={x:Static
local:InverseBooleanConverter.Instance}, Mode=TwoWay}" Margin="0,0,20,0"/>
<RadioButton Content="pdf" IsChecked="{Binding IsPdfFormat}" />
</StackPanel>
</Grid>
```

```
<Button Content="{DynamicResource Str_AddModule}" Command="{Binding CreateNewModuleCommand}"
Grid.Column="2" Padding="15,0" Height="26" VerticalAlignment="Top" Margin="0,1,5,0"/>
</Grid>
</GroupBox>
```

```
<!-- 3. БЛОК РАБОЧЕЙ ОБЛАСТИ (ДЕРЕВО И ПРАВИЛА) -->
<Grid Grid.Row="2" Margin="0,0,0,10">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="1.3*/>
<ColumnDefinition Width="Auto"/>
<ColumnDefinition Width="2*/>
</Grid.ColumnDefinitions>
```

```
<GroupBox Header="{DynamicResource Str_FilesHeader}" Grid.Column="0" Margin="0,0,5,0">
<TreeView ItemsSource="{Binding RootNode.Children}">
```

```

<TreeView.Resources>
<HierarchicalDataTemplate DataType="{x:Type local:FileSystemNode}" ItemsSource="{Binding
Children}">
<StackPanel Orientation="Horizontal" Margin="2">
<CheckBox IsChecked="{Binding IsChecked}" VerticalAlignment="Center" Margin="0,0,6,0"/>
<TextBlock Text="{Binding Name}" VerticalAlignment="Center" Foreground="{DynamicResource
TextBrush}"/>
</StackPanel>
</HierarchicalDataTemplate>
</TreeView.Resources>
</TreeView>
</GroupBox>

<GridSplitter Grid.Column="1" Width="5" HorizontalAlignment="Stretch" Background="{DynamicResource
SplitterBrush}"/>

<GroupBox Header="{DynamicResource Str_PromptHeader}" Grid.Column="2" Margin="5,0,0,0">
<TextBox Text="{Binding PromptRules, UpdateSourceTrigger=PropertyChanged}"
AcceptsReturn="True" TextWrapping="Wrap" VerticalScrollBarVisibility="Auto" BorderThickness="0"/>
</GroupBox>
</Grid>

<!-- 4. БЛОК НИЖНИХ КНОПОК -->
<Grid Grid.Row="3" Margin="0,5,0,0">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="1.3*"/>
<ColumnDefinition Width="2*"/>
</Grid.ColumnDefinitions>

<Button Content="{DynamicResource Str_SaveProject}" Command="{Binding SaveProjectCommand}"
Grid.Column="0" Height="48" Margin="0,0,5,0" Style="{x:Null}" Background="#4CAF50"
Foreground="White" FontWeight="Bold" FontSize="13"/>
<Button Content="{DynamicResource Str_Generate}" Command="{Binding GenerateContextCommand}"
Grid.Column="1" Height="48" Margin="5,0,0,0" Style="{x:Null}" Background="#007ACC"
Foreground="White" FontWeight="Bold" FontSize="13"/>
</Grid>

<!-- ОБЕРПЛЕЙ ПРОГРЕССА -->
<Grid Grid.Row="0" Grid.RowSpan="4" x:Name="ProgressOverlay"
Visibility="{Binding IsProcessing, Converter={StaticResource BooleanToVisibilityConverter},
FallbackValue=Collapsed}">
<Border Background="#A0000000"/>
<Border Width="500" Height="180" CornerRadius="5" VerticalAlignment="Center"
HorizontalAlignment="Center"
Background="{DynamicResource ControlBackgroundBrush}"
BorderBrush="{DynamicResource BorderBrush}" BorderThickness="1" Padding="20">
<Grid>
<Grid.RowDefinitions>
<RowDefinition Height="Auto"/>
<RowDefinition Height="Auto"/>
<RowDefinition Height="*/>
<RowDefinition Height="Auto"/>
</Grid.RowDefinitions>

<TextBlock Text="{Binding ProgressText}" Grid.Row="0" FontSize="14" FontWeight="Bold"
Margin="0,0,0,5" HorizontalAlignment="Center" TextWrapping="NoWrap"/>
<TextBlock Text="{Binding CurrentFileText}" Grid.Row="1" FontSize="11" Margin="0,0,0,15"
Foreground="#A0A0A0" HorizontalAlignment="Center" TextTrimming="CharacterEllipsis"
TextWrapping="NoWrap"/>

<ProgressBar Grid.Row="2" Height="16" VerticalAlignment="Top"
Value="{Binding ProgressValue}" Maximum="{Binding ProgressMax}"
Background="{DynamicResource TextBoxBackgroundBrush}" BorderBrush="{DynamicResource BorderBrush}"/>

<Button Content="{DynamicResource Str_Cancel}" Grid.Row="3"
Command="{Binding CancelGenerationCommand}" Height="32" Width="180" Style="{x:Null}"
Background="#D32F2F" Foreground="White" FontWeight="Bold"/>
</Grid>
</Border>
</Grid>
</Grid>
</Window>

```

```

---MainWindow.xaml.cs---
using System.Collections.ObjectModel;
using System.ComponentModel;
using System.Globalization;
using System.IO;
using System.Text;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Navigation;
using System.Windows.Shapes;

namespace ContextSlicer
{
    /// <summary>
    /// Interaction logic for MainWindow.xaml
    /// </summary>
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            // 1. Сначала принудительно инициализируем язык интерфейса
            InitializeLanguage();

            // 2. Инициализируем компоненты формы и ViewModel
            InitializeComponent();
            DataContext = new MainViewModel();
        }

        private void InitializeLanguage()
        {
            // Определяем язык операционной системы Windows
            string currentCulture = CultureInfo.CurrentUICulture.TwoLetterISOLanguageName;

            // Выбираем нужный файл словаря (по умолчанию для всех – английский, для ru – русский)
            string languageFile = currentCulture.Equals("ru", StringComparison.OrdinalIgnoreCase)
                ? "Strings.ru.xaml"
                : "Strings.en.xaml";

            try
            {
                var dict = new ResourceDictionary
                {
                    Source = new Uri($"pack://application:,,,/Localization/{languageFile}", UriKind.Absolute)
                };

                // Добавляем строковые ресурсы напрямую в контекст этого окна
                this.Resources.MergedDictionaries.Add(dict);

                // Синхронизируем заголовок окна из загруженного словаря
                if (this.Resources.Contains("Str_AppTitle"))
                {
                    this.Title = this.Resources["Str_AppTitle"] as string;
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show($"Failed to load language file: {ex.Message}", "Language Error",
                    MessageBoxButton.OK, MessageBoxImage.Warning);
            }
        }
    }
}

```

```

private void Window_Closing(object sender, System.ComponentModel.CancelEventArgs e)
{
    if (DataContext is MainViewModel vm)
    {
        vm.OnWindowClosing();
    }
}
}
}

```

---models.cs---

```

using CommunityToolkit.Mvvm.ComponentModel;
using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

```

```

namespace ContextSlicer
{
    public class ProjectSettings
    {
        public string ProjectName { get; set; } = string.Empty;
        public string RootPath { get; set; } = string.Empty;
        public string OutputPath { get; set; } = string.Empty;
        public string GlobalRules { get; set; } = string.Empty;
        public List<string> SelectedFiles { get; set; } = new();
    }
}

```

```

public class FileNode : ObservableObject
{
    // Добавляем = string.Empty; чтобы убрать предупреждения компилятора
    public string Name { get; set; } = string.Empty;
    public string FullPath { get; set; } = string.Empty;
}

```

```

public bool IsDirectory { get; set; }

```

```

private bool? _isChecked = false;
public bool? IsChecked
{
    get => _isChecked;
    set => SetProperty(ref _isChecked, value);
}

```

```

public ObservableCollection<FileNode> Children { get; set; } = new();
}

```

```

}

```

---ProjectConfig.cs---

```

using System.Collections.Generic;

```

```

namespace ContextSlicer
{
    // Класс для конкретного нарезанного модуля
    public class ContextModule
    {
        public string ModuleName { get; set; } = string.Empty;
        public string ContextFileName { get; set; } = string.Empty;
        // Список относительных путей файлов, отмеченных ИМЕННО ДЛЯ ЭТОГО модуля
        public List<string> CheckedFiles { get; set; } = new List<string>();
    }
}

```

```

// Класс самого проекта (корневой папки)

```

```
public class ProjectConfig
{
public string ProjectName { get; set; } = string.Empty;
public string RootPath { get; set; } = string.Empty;
public string OutputPath { get; set; } = string.Empty;
public string PromptRules { get; set; } = string.Empty;

// Список всех нарезанных модулей внутри проекта
public List<ContextModule> Modules { get; set; } = new List<ContextModule>();
}
}
```